richfield.ac.za

FACULTY OF INFORMATION TECHNOLOGY

MOBILE APP DEVELOPMENT 700

PRACTICAL ASSIGNMENT REPORT

SMART PANTRY MANAGER

Name and Surname	Mphoentle Thomas
Student ITS No.	402411462
Module	Mobile App Development 700
GitHub repository	https://github.com/Saucydotnineseven/SmartPantry
Year of Study / Qualification / Semester	[Complete these details]
Date submitted	[Enter submission date]
Assignment due date	[Enter due date]

Open the Smart Pantry Manager GitHub repository

DECLARATION OF ORIGINALITY

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: _____ Date: _____

Please complete the remaining cover details and sign the declaration before submission.

TABLE OF CONTENTS

No table of contents entries found.

If the contents list does not populate automatically, click it in Word and choose Update Field → Update entire table.

1. INTRODUCTION

Smart Pantry Manager is a Java Android application designed to help people use ingredients they already have at home. Food can be wasted when people lose track of what is in the pantry or do not know what they can prepare from the available ingredients. The app stores pantry items and compares them with a local collection of recipes.

The central requirement is strict recipe matching: a recipe is suggested only when every required ingredient is present in a sufficient quantity. A partial match is not shown as a recipe the user can make. The app works offline and stores pantry data on the device using SQLite.

2. SYSTEM DESIGN

2.1 Screen flow

The pantry is the starting screen. The user can add an ingredient, tap an existing row to edit it, or long-press a row to delete it. The Recipes action opens the strict suggestions list; selecting a recipe opens its detail screen. Settings is available from the bottom navigation.

Screen	User action / next screen
Pantry List	Add → Add/Edit Ingredient; select row → Edit; long-press → Delete confirmation; Recipes → Suggested Recipes; Settings → Settings
Add/Edit Ingredient	Validate fields → save to SQLite → return to Pantry List
Suggested Recipes	Select a fully matched recipe → Recipe Detail; Back → Pantry List
Recipe Detail	Read ingredients and numbered method → Back to Suggested Recipes
Settings	Save expiry alert and unit preferences → Back to Pantry List

2.2 Data model

The local SQLite database has three related tables. A pantry row stores a user-owned ingredient. A recipe row stores its name and method. Recipe ingredients connect the catalogue recipes to their required quantities and units.

PANTRY	RECIPES	RECIPE_INGREDIENTS
id (PK) name quantity unit expiry_date	id (PK) name steps	id (PK) recipe_id (FK) name quantity unit
One row per pantry item	One row per recipe	Many ingredient rows belong to one recipe

Relationship: RECIPES.id (one) → RECIPE_INGREDIENTS.recipe_id (many). Pantry items are stored independently because they represent the user's current stock; the matcher compares them with the recipe ingredient rows.

2.3 Main components

Component	Responsibility
DatabaseHelper.java	Creates/upgrades SQLite tables, seeds the recipe catalogue, and provides pantry CRUD and recipe reads.
PantryAdapter.java	Binds pantry records to RecyclerView rows and forwards tap/long-press actions.
RecipeMatcher.java	Normalizes basic singular/plural differences, converts supported units, totals pantry stock, and applies the strict all-ingredients check.
Activities and Intents	MainActivity, AddEditActivity, SuggestedRecipesActivity, RecipeDetailActivity, and SettingsActivity implement the app screens and pass item/recipe identifiers between screens.

3. SCREENSHOTS OF THE RUNNING APP

The figures below use screenshots captured from the running app on the phone. Figure 8 remains a placeholder for the recipe detail screen; keep the full screen visible when adding it.

Figure 1. Pantry list

Populated pantry and bottom navigation.

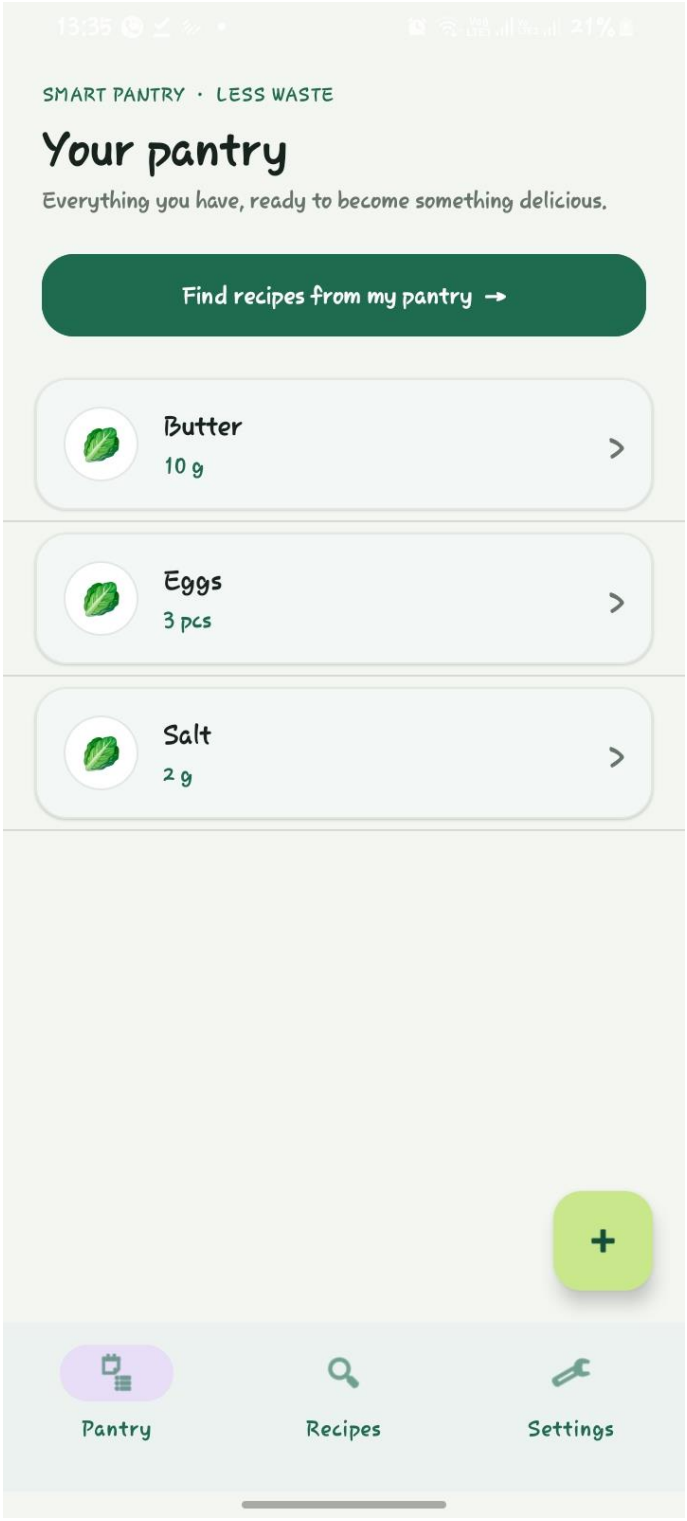


Figure 2. Add ingredient form

The blank form before saving a new pantry item.

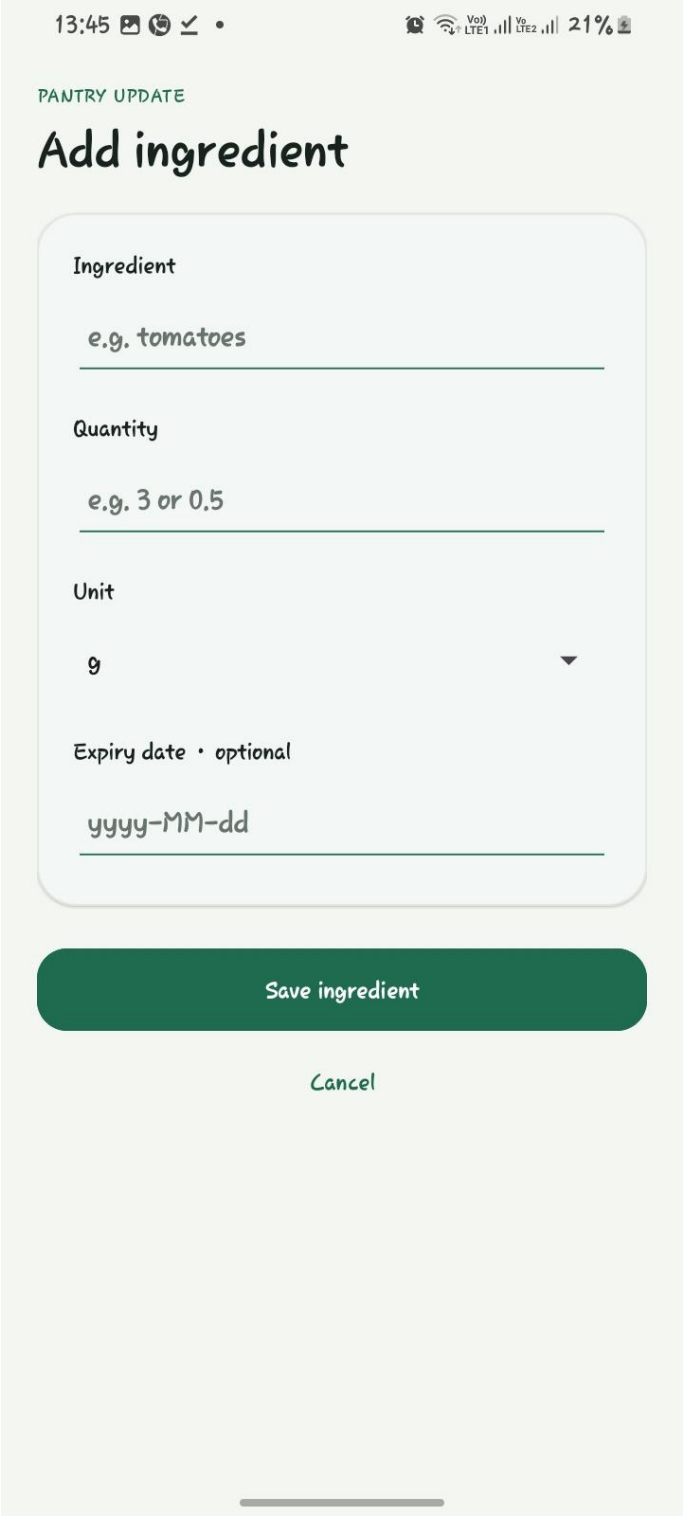


Figure 3. Input validation

The invalid expiry date displays the form validation message.

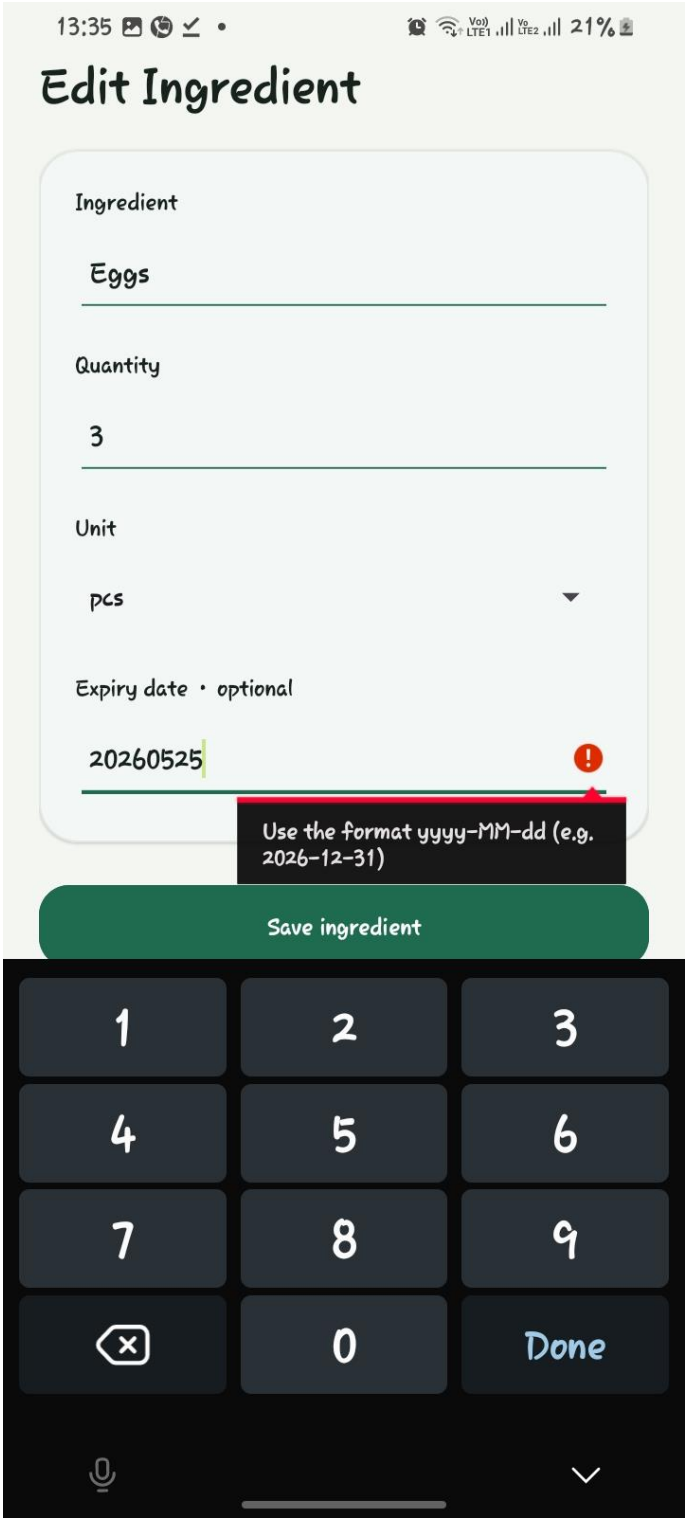


Figure 4. Edit ingredient

An existing item is open for editing with its saved name, quantity, and unit.

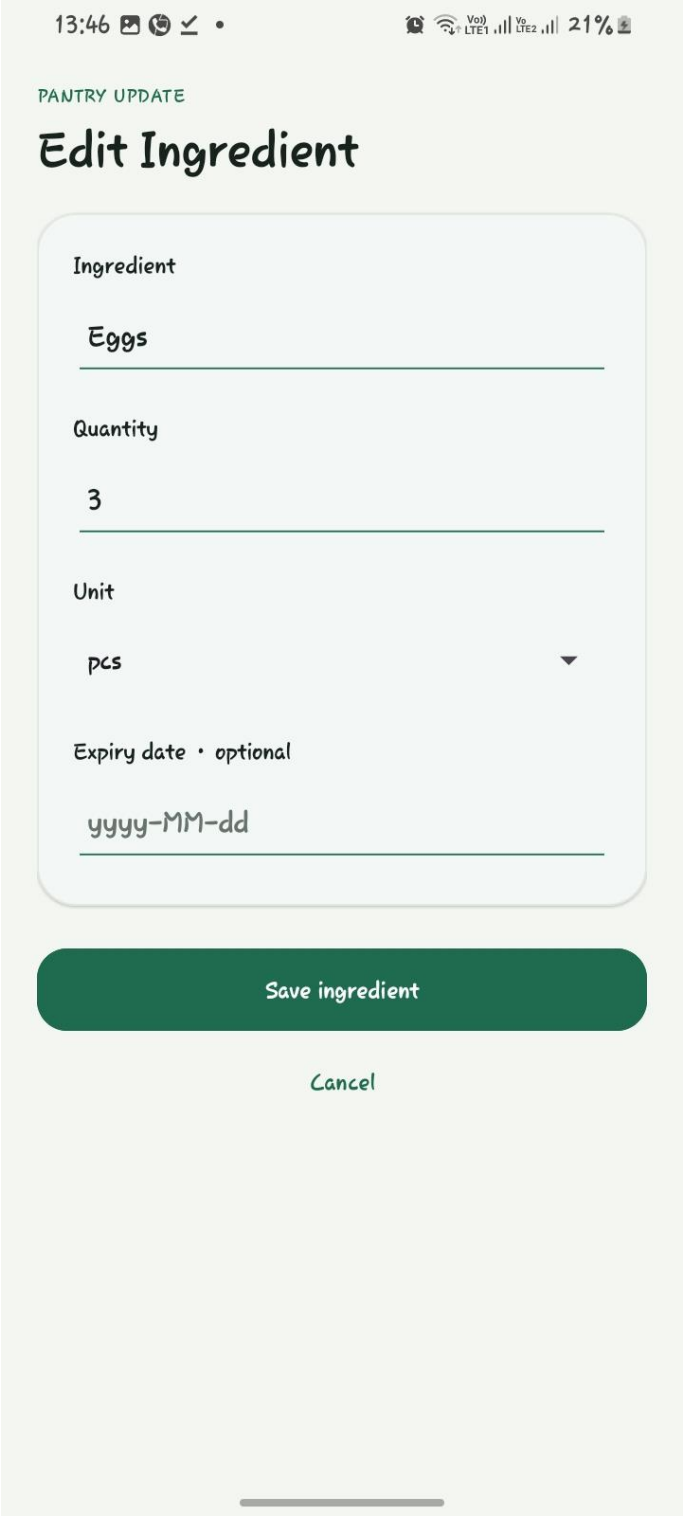


Figure 5. Delete confirmation

Long-pressing a pantry item opens this confirmation before deletion.

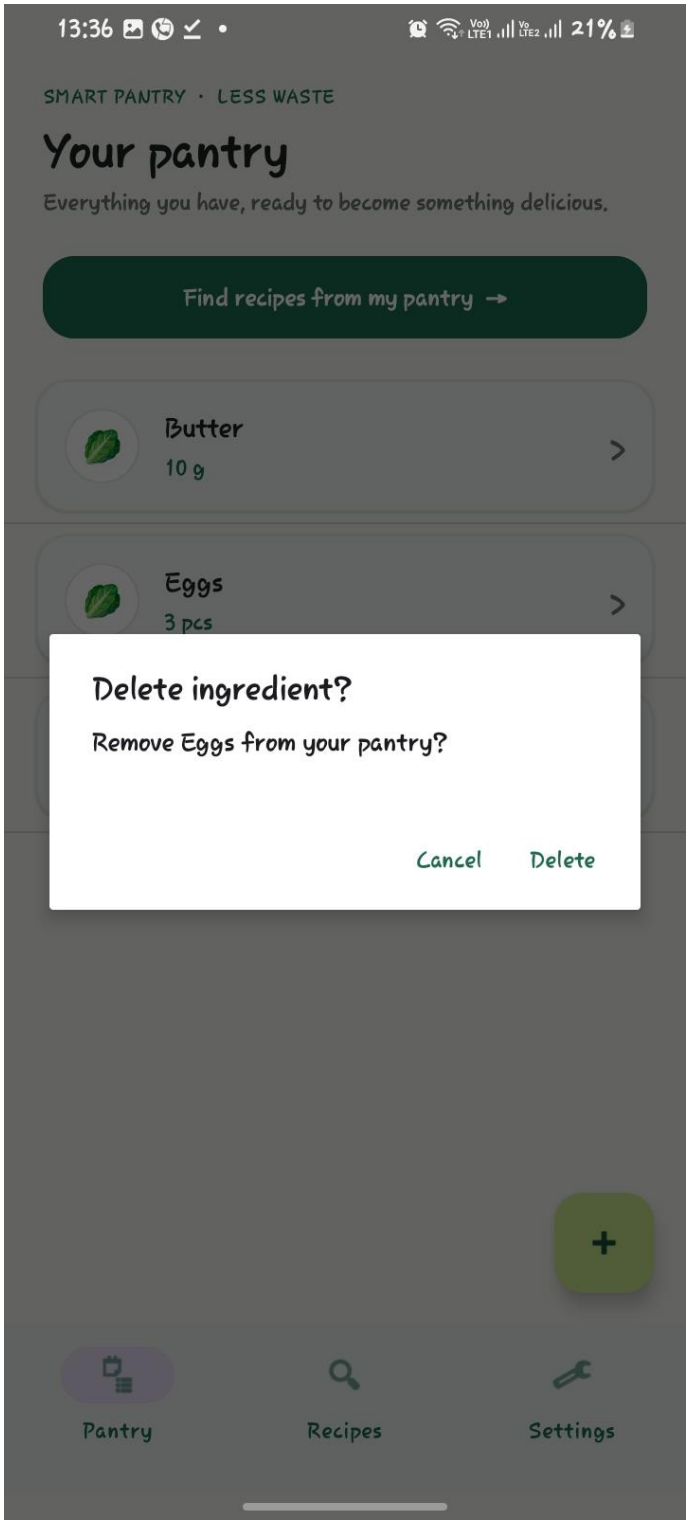


Figure 6. Strict recipe suggestions

Scrambled Eggs appears when eggs, butter, and salt meet the required quantities.

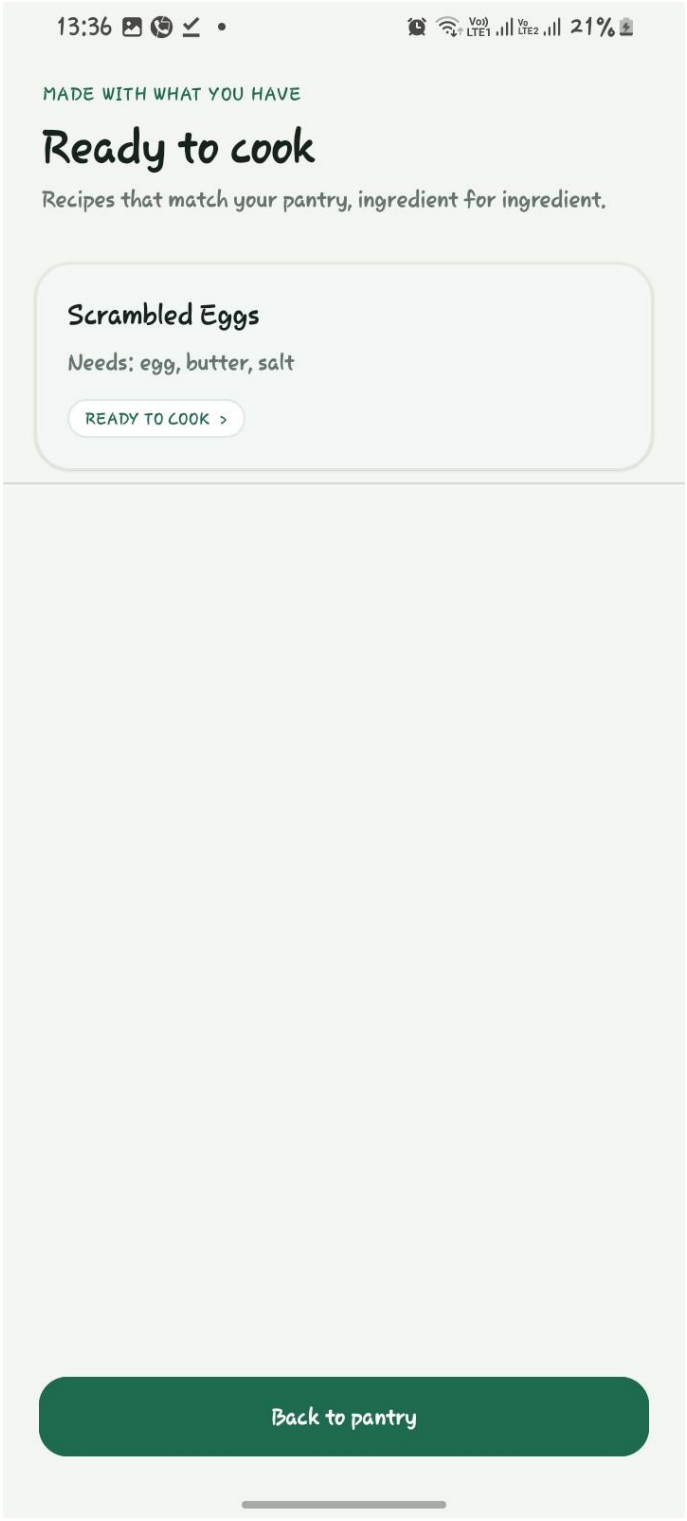


Figure 7. Missing ingredient check

No-match state after removing or reducing a required ingredient.

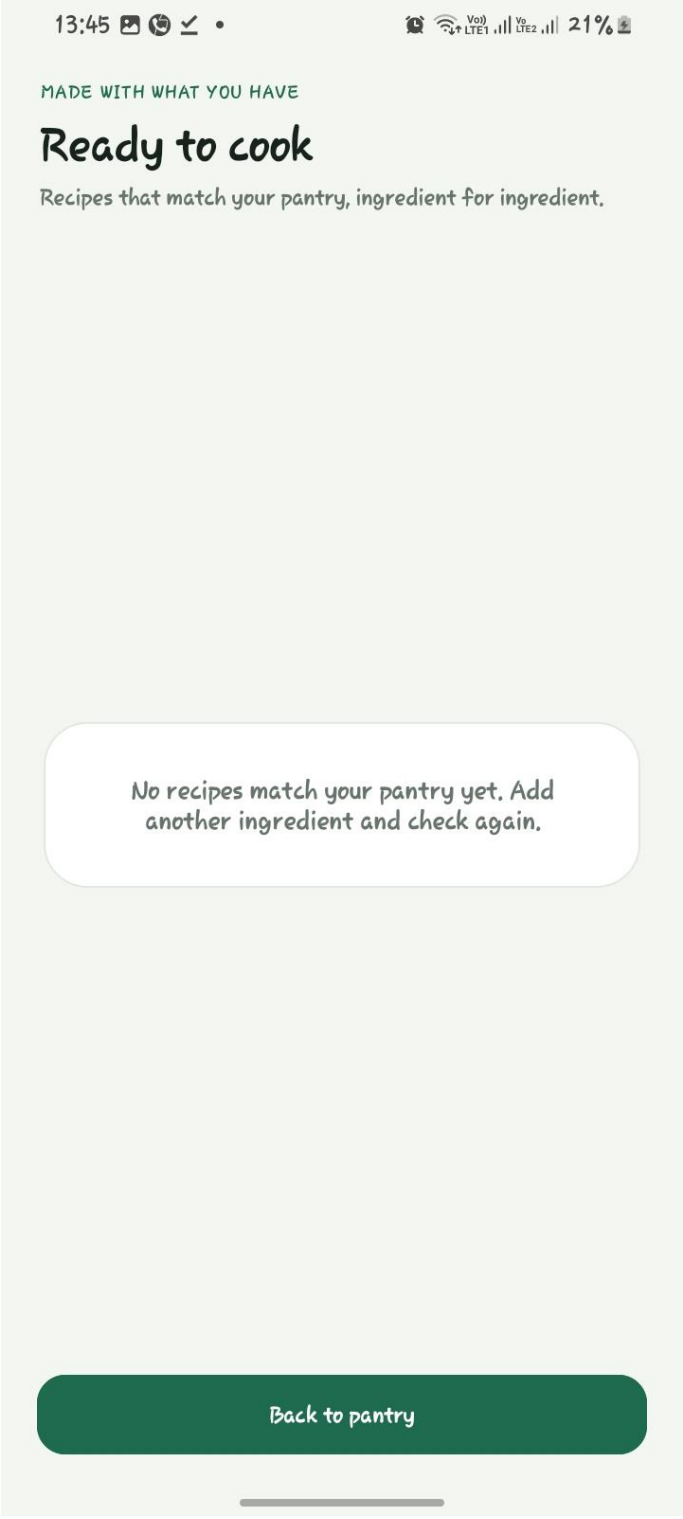
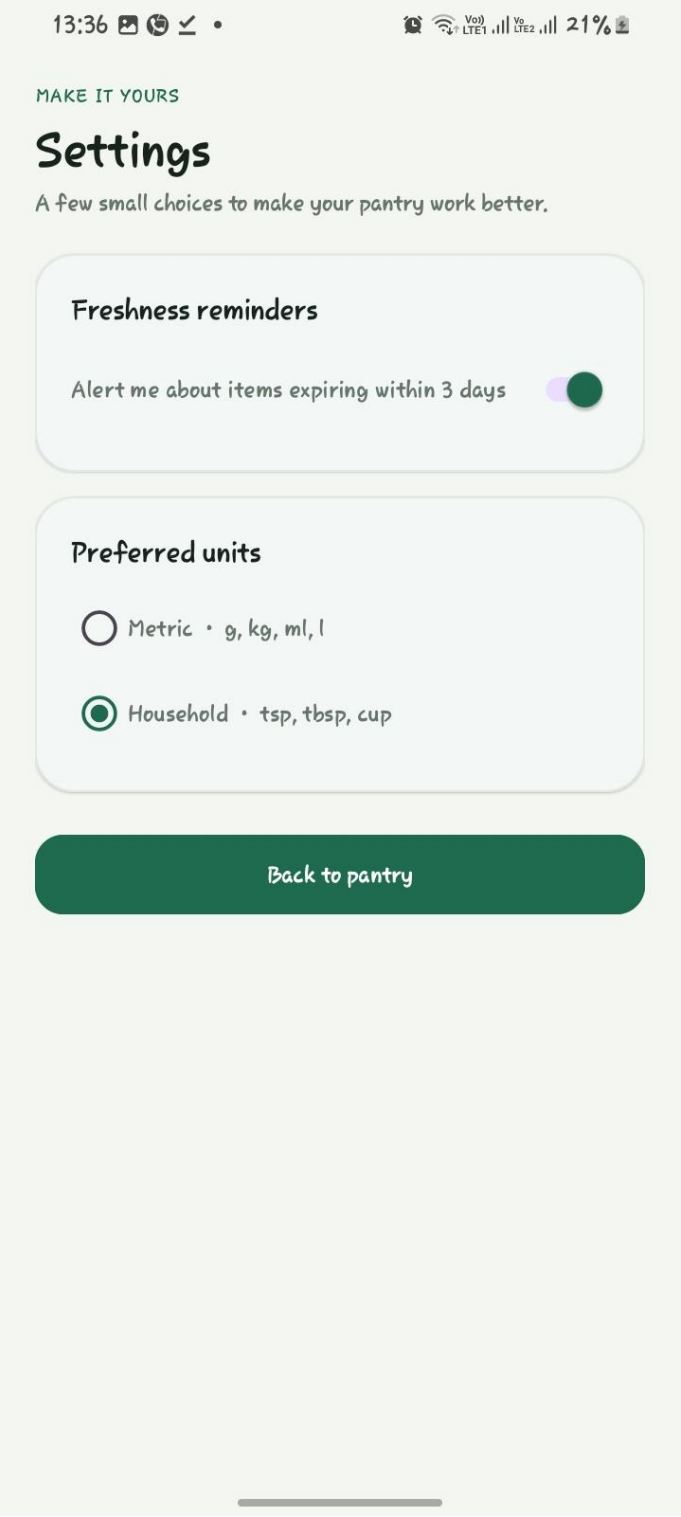


Figure 8. Recipe detail
Capture a selected recipe's complete ingredient list and numbered method.
PASTE REAL PHONE SCREENSHOT HERE

Figure 9. Settings

Saved expiry alert and unit preference controls.



4. KEY CODE SNIPPETS

The following short examples are taken from the project source and highlight the main Android and business-logic concepts.

4.1 Strict matching — RecipeMatcher.java

```
for (RecipeIngredient ing : recipe.getIngredients()) {  String key =
makeKey(ing.getName(), ing.getUnit());  double needed =
toBaseAmount(ing.getQuantity(), ing.getUnit());  Double have = available.get(key);  if
(have == null || have + 0.0001 < needed) return false; } return true;
```

The matcher checks every required ingredient. A missing item or insufficient quantity immediately rejects the recipe; only after all rows pass does the method return true.

4.2 Pantry data persistence — DatabaseHelper.java

```
public long addPantryItem(PantryItem item) {  SQLiteDatabase db =
getWritableDatabase();  ContentValues values = new ContentValues();
values.put(COL_NAME, item.getName());  values.put(COL_QTY, item.getQuantity());
values.put(COL_UNIT, item.getUnit());  values.put(COL_EXPIRY,
item.getExpiryDate());  return db.insert(TABLE_PANTRY, null, values); }
```

The helper maps a PantryItem to database columns and inserts it into SQLite. Related methods read, update, and delete pantry rows, so user data persists between app sessions.

4.3 Activity navigation — MainActivity.java

```
Intent intent = new Intent(MainActivity.this, AddEditActivity.class);
intent.putExtra(AddEditActivity.EXTRA_ITEM_ID, item.getId()); startActivity(intent);
```

An explicit Intent opens the add/edit activity. When editing, the pantry item's ID is passed as an extra so the destination can load and update the correct database row.

4.4 RecyclerView refresh — MainActivity.java

```
protected void onResume() {  super.onResume();  loadPantry();
bottomNav.setSelectedItemId(R.id.nav_pantry);  showExpiryAlert(); }
```

The pantry is reloaded when MainActivity becomes visible again, so an item added or edited on another screen appears in the list when the user returns.

5. CHALLENGES AND SOLUTIONS

The examples below are based on the implementation and debugging work completed in this project. Please confirm they match your own experience and edit them in your own words before submission.

Challenge 1: A strict matcher must compare quantities even when users enter common unit variants or plural names. Solution: RecipeMatcher normalizes case and simple plurals, converts supported weight/volume units to a base unit, and only accepts a recipe after every ingredient passes.

Challenge 2: A recipe catalogue created only during database initialization can be missing on an already-installed app if initialization logic changes. Solution: The database version was advanced and the upgrade path now creates missing tables and seeds recipes only when the catalogue is empty, while retaining pantry rows.

Add one more genuine challenge you encountered, what caused it, and how you solved it: [Complete in your own words.]

6. CONCLUSION AND REFLECTION

Smart Pantry Manager brings together Android activities, explicit intents, RecyclerView adapters, input validation, SharedPreferences, and SQLite persistence to solve a focused food-waste problem. Its key behavior is the strict recipe matcher: suggestions are limited to recipes whose required ingredients and quantities are all available in the pantry.

Personal reflection — complete this paragraph with what you learned, which part you found most challenging, and one improvement you would make with more time: [Write your reflection here.]

7. REFERENCES

Richfield Graduate Institute of Technology (2026) Mobile App Development 700: Practical Assignment — Smart Pantry Manager. Assignment brief supplied for the module.

Google (n.d.) SQLiteOpenHelper. Android Developers API reference. Available at: https://developer.android.com/reference/android/database/sqlite/SQLiteOpenHelper (Accessed: 4 November 2026).

Google (n.d.) Create dynamic lists with RecyclerView. Android Developers. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview (Accessed: 4 November 2026).

Google (n.d.) Intents and intent filters. Android Developers. Available at: https://developer.android.com/guide/components/intents-filters (Accessed: 4 November 2026).

Smart Pantry Manager source code (2026) Java Android project. GitHub repository: https://github.com/Saucydotnineseven/SmartPantry (Accessed: 4 November 2026).